Module 7 Notes: Singly Linked List Insertion by Position

1. Current progress analysis

You are ready for Module 7 because you already know the ideas needed to understand insertion.

From Module 1

You learned why linked lists exist.

Arrays are fast when we know the index, but arrays have fixed size after creation. Linked lists are more flexible because they can grow and shrink one node at a time.

Structure	Strength	Weakness
Array	Fast direct access by index	Fixed size and shifting may be needed during insertion/deletion
Linked list	Flexible size and easier pointer-based insertion/deletion	No direct jumping to an index

This matters because insertion in an array often needs shifting, but insertion in a linked list mainly needs pointer changes.

From Module 2

You learned what a node is.

A singly linked-list node has two parts:

```
[data | next]
```

Part	Meaning
`data`	The value stored in the node
`next`	The address of the next node

C node definition:

```c
struct Node {
    int data;
```

```
    struct Node *next;
};
```

You also learned the key pointer movement:

```
p = p->next;
```

This moves pointer `p` to the next node.

From Module 3

You learned traversal and list creation.

Traversal means visiting nodes one by one:

```
first -> 8 -> 3 -> 7 -> 12 -> NULL
```

Basic display pattern:

```
void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}
```

Important rule:

> Do not move `first` during normal traversal. Use a temporary pointer like `p`.

This matters in Module 7 because insertion sometimes needs to move through the list to reach the correct position.

From Module 4

You learned recursion and the base case:

```
p == NULL
```

This means the list has ended.

Module 7 is mostly iterative, but the same idea of checking `NULL` still matters for safety.

From Module 5

You learned count, sum, max, and min.

The most important function from Module 5 for this module is `count()` because Module 7 uses it to validate insertion indexes.

From Module 6

You learned searching and move-to-head.

Move-to-head was important because it changed links:

```
q->next = p->next;
p->next = first;
first = p;
```

Module 7 continues that idea. Instead of only reading nodes, we now change links to insert a new node.

---

2. Big idea of Module 7

Module 7 teaches how to insert a new node into a singly linked list at a given position.

Insertion means:

> Create a new node and connect it into the linked list without losing the existing nodes.

Example list:

```
8 -> 3 -> 7 -> 12 -> NULL
```

If we insert `10` at the beginning, the result should be:

```
10 -> 8 -> 3 -> 7 -> 12 -> NULL
```

If we insert `10` after `7`, the result should be:

```
8 -> 3 -> 7 -> 10 -> 12 -> NULL
```

The main idea is:

> Linked-list insertion does not shift elements. It changes pointers.

In an array, inserting may require moving many values. In a linked list, we create a new node and adjust the `next` links.

---

3. The insertion index convention

This module uses a specific index convention.

```
index 0 = insert before the first node
index 1 = insert after the first node
index 2 = insert after the second node
index 3 = insert after the third node
...
```

Example list:

```
8 -> 3 -> 7 -> 12 -> NULL
```

This list has 4 nodes.

The valid insertion indexes are:

```
0, 1, 2, 3, 4
```

Meaning:

Index	Meaning	Result if inserting 10
`0`	Insert before 8	`10 -> 8 -> 3 -> 7 -> 12 -> NULL`
`1`	Insert after 8	`8 -> 10 -> 3 -> 7 -> 12 -> NULL`
`2`	Insert after 3	`8 -> 3 -> 10 -> 7 -> 12 -> NULL`
`3`	Insert after 7	`8 -> 3 -> 7 -> 10 -> 12 -> NULL`
`4`	Insert after 12	`8 -> 3 -> 7 -> 12 -> 10 -> NULL`

If the list has `n` nodes, valid insertion indexes are:

```
0 to n
```

Invalid indexes include:

```
-1
n + 1
100
```

For example, if the list has 4 nodes, index `5` is invalid.

---

4. Why index 0 is special

Index `0` means inserting before the first node.

This is special because the first node of the list is controlled by the pointer `first`.

Suppose the list is:

```
first
 |
 v
8 -> 3 -> 7 -> NULL
```

If we insert `10` at index `0`, the new node must become the first node.

Final result:

```
first
 |
 v
10 -> 8 -> 3 -> 7 -> NULL
```

So we must update `first`.

For any other index, we do not move `first`. We only change the `next` pointer of some node inside the list.

---

5. Case 1: Inserting at index 0

Suppose the list is:

```
first
 |
 v
8 -> 3 -> 7 -> NULL
```

We want to insert `10` at index `0`.

Step 1: Create a new node

```
struct Node *t;
t = (struct Node *)malloc(sizeof(struct Node));
t->data = 10;
```

Now we have a new node:

```
t
|
v
[10 | ?]
```

The `next` part is not set yet.

Step 2: Make the new node point to the old first node

```
t->next = first;
```

Now:

```
t
|
v
10 -> 8 -> 3 -> 7 -> NULL
      ^
      |
   first
```

The new node points to the old list.

Step 3: Move `first` to the new node

```
first = t;
```

Final result:

```
first
 |
 v
10 -> 8 -> 3 -> 7 -> NULL
```

Head insertion code

```
t->next = first;
first = t;
```

This is the key pattern for insertion at the beginning.

Why this order is safe

We first connect the new node to the old list:

```
t->next = first;
```

Then we update `first`:

```
first = t;
```

If we changed `first` first without saving the old list, we could lose access to the original first node.

---

6. Case 2: Inserting after a position

Suppose the list is:

```
8 -> 3 -> 7 -> 12 -> NULL
```

We want to insert `10` at index `3`.

Using this module's convention:

```
index 3 = insert after the third node
```

The third node is `7`.

So the final list should be:

```
8 -> 3 -> 7 -> 10 -> 12 -> NULL
```

Step 1: Move `p` to the node before the insertion point

For index `3`, `p` must stop at the third node.

```
8 -> 3 -> 7 -> 12 -> NULL
          ^
          |
          p
```

That means `p` points to `7`.

Step 2: Create the new node

```c
struct Node *t;
t = (struct Node *)malloc(sizeof(struct Node));
t->data = 10;
```

New node:

```
t
|
v
10
```

Step 3: Make the new node point to the node after `p`

```
t->next = p->next;
```

Before this line:

```
p
|
v
7 -> 12 -> NULL

t
|
v
10
```

After this line:

```
10 -> 12 -> NULL
```

So the new node now points to the node that originally came after `p`.

Step 4: Make `p` point to the new node

```
p->next = t;
```

Now `7` points to `10`.

Final result:

```
8 -> 3 -> 7 -> 10 -> 12 -> NULL
```

Middle/end insertion code pattern

```
t->next = p->next;
p->next = t;
```

This is the heart of insertion after a position.

---

7. Why the order of link changes matters

The correct order is:

```
t->next = p->next;
p->next = t;
```

This means:

1. New node points to the old next node.
2. Previous node points to the new node.

Correct example

Original:

```
7 -> 12 -> NULL
^
|
p

10
^
|
t
```

Do this first:

```
t->next = p->next;
```

Now:

```
10 -> 12 -> NULL
```

Then:

```
p->next = t;
```

Now:

```
7 -> 10 -> 12 -> NULL
```

Correct final result.

Wrong order

Wrong code:

```
p->next = t;
t->next = p->next;
```

Why is this wrong?

After this line:

```
p->next = t;
```

`p->next` no longer points to the old next node. It now points to `t`.

Then this line:

```
t->next = p->next;
```

makes `t->next` point to `t` itself.

That can create a self-loop and lose the rest of the list.

Important rule:

> Copy the old link before overwriting it.

So always use:

```
t->next = p->next;
p->next = t;
```

---

8. How to move `p` to the correct position

For non-head insertion, we need `p` to stop at the node before the new node.

The loop is:

```
for (int i = 0; i < index - 1; i++) {
    p = p->next;
}
```

Why `index - 1`?

Because `p` starts at the first node.

Example:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Insert at index `3`.

We want `p` to stop at the third node, which is `7`.

Start:

```
p points to 8
```

Loop movements:

Move	`p` points to
Start	8
Move 1	3
Move 2	7

For index `3`, we move `2` times.

That is why the loop runs `index - 1` times.

---

9. Count function for validation

Before inserting, we should check whether the index is valid.

To do that, we need to know how many nodes are in the list.

The `count()` function counts the nodes:

```
int count(struct Node *p) {
    int c = 0;

    while (p != NULL) {
        c++;
        p = p->next;
    }

    return c;
}
```

How `count()` works

Example list:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Dry run:

Step	`p` points to	Action	`c`
Start	8	Start count	0
1	8	Count node	1
2	3	Count node	2
3	7	Count node	3
4	12	Count node	4
5	NULL	Stop	4

Final count:

```
4
```

---

10. Index validation rule

If a list has `n` nodes, valid insertion indexes are:

```
0 to n
```

So the validation rule is:

```
if (index < 0 || index > count(p)) {
    return;
}
```

This means:

Condition	Meaning
`index < 0`	Negative indexes are invalid
`index > count(p)`	Index is beyond the allowed range

Example:

List:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Count is `4`.

Valid indexes:

```
0, 1, 2, 3, 4
```

Invalid indexes:

```
-1, 5, 6, 100
```

Why validation matters:

If we try to insert at an invalid position, pointer `p` may become `NULL`. Then using `p->next` would be unsafe and may crash the program.

---

11. Complete Insert function

```
void Insert(struct Node *p, int index, int x) {
    struct Node *t;

    if (index < 0 || index > count(p)) {
        return;
    }

    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;

    if (index == 0) {
        t->next = first;
        first = t;
    } else {
        p = first;

        for (int i = 0; i < index - 1; i++) {
            p = p->next;
        }

        t->next = p->next;
        p->next = t;
    }
}
```

Function header

```
void Insert(struct Node *p, int index, int x)
```

Meaning:

Parameter	Meaning
`p`	Pointer used to traverse the list

Parameter	Meaning
`index`	Position where the new node should be inserted
`x`	Value to store in the new node

The function returns `void` because it directly changes the linked list.

Local pointer `t`

```
struct Node *t;
```

`t` is a temporary pointer for the new node.

Important:

```
struct Node *t;
```

only creates a pointer.

This actually creates the node:

```
t = (struct Node *)malloc(sizeof(struct Node));
```

Validation

```
if (index < 0 || index > count(p)) {
    return;
}
```

If the index is invalid, the function stops immediately.

Create and fill the new node

```
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
```

This creates a new node and stores `x` inside it.

Index 0 case

```
if (index == 0) {
    t->next = first;
    first = t;
}
```

This inserts before the old first node.

Non-zero index case

```
else {
    p = first;

    for (int i = 0; i < index - 1; i++) {
        p = p->next;
    }

    t->next = p->next;
    p->next = t;
}
```

This moves `p` to the node before the insertion point, then inserts the new node after `p`.

---

12. Full working C program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void create(int A[], int n) {
    if (n <= 0) {
        first = NULL;
        return;
    }
```

```
    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;

        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

int count(struct Node *p) {
    int c = 0;

    while (p != NULL) {
        c++;
        p = p->next;
    }

    return c;
}

void Insert(struct Node *p, int index, int x) {
    struct Node *t;

    if (index < 0 || index > count(p)) {
        return;
    }

    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;

    if (index == 0) {
        t->next = first;
```

```
        first = t;
    } else {
        p = first;

        for (int i = 0; i < index - 1; i++) {
            p = p->next;
        }

        t->next = p->next;
        p->next = t;
    }
}

int main() {
    int A[] = {8, 3, 7, 12};

    create(A, 4);

    printf("Original list: ");
    display(first);

    Insert(first, 0, 10);
    printf("\nAfter inserting 10 at index 0: ");
    display(first);

    Insert(first, 3, 99);
    printf("\nAfter inserting 99 at index 3: ");
    display(first);

    Insert(first, count(first), 50);
    printf("\nAfter inserting 50 at the end: ");
    display(first);

    return 0;
}
```

Possible output:

```
Original list: 8 3 7 12
After inserting 10 at index 0: 10 8 3 7 12
After inserting 99 at index 3: 10 8 3 99 7 12
After inserting 50 at the end: 10 8 3 99 7 12 50
```

---

13. Dry run 1: Insert at index 0

Original list:

```
8 -> 3 -> 7 -> NULL
```

Call:

```
Insert(first, 0, 10);
```

Step 1: Validate index

Count is `3`.

Valid indexes are:

```
0, 1, 2, 3
```

Index `0` is valid.

Step 2: Create new node

```
t->data = 10;
```

New node:

```
10
```

Step 3: Connect new node to old first

```
t->next = first;
```

Now:

```
10 -> 8 -> 3 -> 7 -> NULL
```

Step 4: Update first

```
first = t;
```

Final list:

```
10 -> 8 -> 3 -> 7 -> NULL
```

---

14. Dry run 2: Insert in the middle

Original list:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Call:

```
Insert(first, 3, 10);
```

Step 1: Validate index

Count is `4`.

Valid indexes:

```
0, 1, 2, 3, 4
```

Index `3` is valid.

Step 2: Create new node

```
10
```

Step 3: Move `p`

Since index is `3`, move `p` `index - 1`, or `2`, times.

Stage	`p` points to
Start	8
Move 1	3
Move 2	7

Now:

```
8 -> 3 -> 7 -> 12 -> NULL
          ^
          |
          p
```

Step 4: Link new node to old next node

```
t->next = p->next;
```

Since `p` points to `7`, `p->next` points to `12`.

Now:

```
10 -> 12 -> NULL
```

Step 5: Link previous node to new node

```
p->next = t;
```

Now:

```
7 -> 10
```

Final list:

```
8 -> 3 -> 7 -> 10 -> 12 -> NULL
```

---

15. Dry run 3: Insert after the first node

Original list:

```
8 -> 3 -> 7 -> NULL
```

Call:

```
Insert(first, 1, 10);
```

Index `1` means insert after the first node.

The loop is:

```
for (int i = 0; i < index - 1; i++) {
    p = p->next;
}
```

Here:

```
index - 1 = 0
```

So the loop does not move `p`.

`p` stays at the first node:

```
8 -> 3 -> 7 -> NULL
^
|
p
```

Then:

```
t->next = p->next;
p->next = t;
```

Final list:

```
8 -> 10 -> 3 -> 7 -> NULL
```

---

16. Dry run 4: Insert at the end

Original list:

```
8 -> 3 -> 7 -> NULL
```

Call:

```
Insert(first, 3, 10);
```

The list has 3 nodes. Index `3` means insert after the third node, which is the end.

Move `p` until it reaches the third node:

Stage	`p` points to
Start	8
Move 1	3
Move 2	7

Now:

```
8 -> 3 -> 7 -> NULL
          ^
          |
          p
```

Since `p` is the last node:

```
p->next == NULL
```

Now:

```
t->next = p->next;
```

means:

```
t->next = NULL;
```

Then:

```
p->next = t;
```

Final list:

```
8 -> 3 -> 7 -> 10 -> NULL
```

---

17. Dry run 5: Invalid index

Original list:

```
8 -> 3 -> 7 -> NULL
```

Call:

```
Insert(first, 5, 10);
```

The list has 3 nodes.

Valid indexes are:

```
0, 1, 2, 3
```

Index `5` is invalid.

The validation catches it:

```
if (index < 0 || index > count(p)) {
    return;
}
```

The function returns without changing the list.

Final list:

```
8 -> 3 -> 7 -> NULL
```

---

18. Empty list insertion

An empty list means:

```
first == NULL
```

The list looks like:

```
NULL
```

If the list is empty, its count is `0`.

Valid insertion indexes are:

```
0
```

Call:

```
Insert(first, 0, 10);
```

Inside the index `0` case:

```
t->next = first;
first = t;
```

Since `first` is `NULL`, this means:

```
t->next = NULL;
first = t;
```

Final list:

```
10 -> NULL
```

So the same index `0` code works for both:

1. inserting before an existing first node
2. inserting into an empty list

---

19. Complexity analysis

Let `n` be the number of nodes in the list.

Head insertion

```
t->next = first;
first = t;
```

This changes only a fixed number of pointers.

Time complexity:

```
O(1)
```

Extra space:

```
O(1)
```

Insertion in the middle

To insert in the middle, we must move `p` to the correct position.

That may require walking through many nodes.

Time complexity:

```
O(n)
```

Extra space:

```
O(1)
```

Insertion at the end

Without a separate tail or `last` pointer, inserting at the end requires reaching the last node.

Time complexity:

```
O(n)
```

Extra space:

```
O(1)
```

Validation cost

If the function uses:

```
count(p)
```

then validation itself takes:

```
O(n)
```

So even if the actual insertion at the head is `O(1)`, validation with `count()` adds an `O(n)` pass.

Important distinction:

Situation	Time
Head insertion without validation	`O(1)`
Head insertion with `count()` validation	`O(n)` because `count()` traverses
Middle/end insertion	`O(n)`

Space complexity

Insertion uses only a few pointer variables:

```
p, t
```

So extra space is:

```
O(1)
```

The new node itself is not counted as extra workspace because creating a node is the purpose of insertion.

---

20. Important pointer patterns to memorize

Create a new node

```
struct Node *t;
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
```

Insert at the head

```
t->next = first;
first = t;
```

Insert after node `p`

```
t->next = p->next;
p->next = t;
```

Move through the list

```
p = p->next;
```

Validate index

```
if (index < 0 || index > count(p)) {
    return;
}
```

---

21. Common beginner mistakes

Mistake 1: Reversing the link order

Wrong:

```
p->next = t;
t->next = p->next;
```

Correct:

```
t->next = p->next;
p->next = t;
```

Why?

The new node must first remember where the old next node was.

Mistake 2: Forgetting that index 0 is special

Wrong idea:

> All insertion cases can use the same loop.

Problem:

Index `0` means inserting before `first`, so `first` must change.

Correct structure:

```
if (index == 0) {
    t->next = first;
    first = t;
```

```
} else {
    // move p and insert after p
}
```

Mistake 3: Not validating the index

Wrong:

```
Insert(first, 100, 10);
```

If the function does not validate, `p` may become `NULL`, and then this is unsafe:

```
p->next
```

Correct:

```
if (index < 0 || index > count(p)) {
    return;
}
```

Mistake 4: Thinking `struct Node *t;` creates a node

This only creates a pointer:

```
struct Node *t;
```

This creates the actual node:

```
t = (struct Node *)malloc(sizeof(struct Node));
```

Mistake 5: Moving `first` by mistake

During traversal, do not do this unless you truly want to change the head:

```
first = first->next;
```

Use a temporary pointer:

```
p = first;
p = p->next;
```

For insertion at positions other than `0`, `first` should stay pointing to the first node.

Mistake 6: Forgetting to set `t->data`

Wrong:

```
t = (struct Node *)malloc(sizeof(struct Node));
t->next = p->next;
p->next = t;
```

Problem:

The node was inserted, but its data was never assigned.

Correct:

```
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
t->next = p->next;
p->next = t;
```

Mistake 7: Confusing index with value

In this call:

```
Insert(first, 3, 10);
```

`3` is the index.

`10` is the value to insert.

It means:

```
Insert value 10 at index 3.
```

It does not mean search for value `3`.

---

22. Practice questions

Question 1

Given:

```
8 -> 3 -> 7 -> NULL
```

What is the result of this call?

```
Insert(first, 0, 10);
```

Answer:

```
10 -> 8 -> 3 -> 7 -> NULL
```

Question 2

Given:

```
8 -> 3 -> 7 -> NULL
```

What is the result of this call?

```
Insert(first, 1, 10);
```

Answer:

```
8 -> 10 -> 3 -> 7 -> NULL
```

Question 3

Given:

```
8 -> 3 -> 7 -> NULL
```

What is the result of this call?

```
Insert(first, 3, 10);
```

Answer:

```
8 -> 3 -> 7 -> 10 -> NULL
```

Question 4

Why is this order correct?

```
t->next = p->next;
p->next = t;
```

Answer:

Because the new node first stores the address of the old next node. Then `p` is changed to point to the new node. This prevents losing the rest of the list.

Question 5

Why is index `0` handled separately?

Answer:

Because index `0` inserts before the first node, so the `first` pointer must be updated.

Question 6

If a list has 5 nodes, what insertion indexes are valid?

Answer:

```
0, 1, 2, 3, 4, 5
```

Question 7

What happens if the index is invalid?

Answer:

The function should return without changing the list.

Question 8

What is the time complexity of inserting at the head without validation?

Answer:

```
O(1)
```

Question 9

What is the time complexity of inserting at the end without a `last` pointer?

Answer:

```
O(n)
```

Because we must traverse to the last node.

Question 10

What is the extra space complexity of insertion?

Answer:

```
O(1)
```

Only a few pointer variables are used.

---

23. Mini checklist for Module 7 mastery

You understand Module 7 if you can do all of these:

- Explain what insertion means in a linked list.
- State the index convention clearly.
- Explain why index `0` is special.
- Write the head insertion lines from memory.
- Write the middle insertion lines from memory.
- Explain why `t->next = p->next` must happen before `p->next = t`.

- Use `count()` to validate an index.
- Dry run insertion at the head.
- Dry run insertion in the middle.
- Dry run insertion at the end.
- Explain the time complexity of each case.
- Avoid moving `first` unless you are intentionally changing the head.

---

24. Final Module 7 summary

Module 7 teaches how to insert a node into a singly linked list by position.

The main idea is:

> Insertion means creating a new node and changing links safely.

The two most important insertion patterns are:

Insert at the head

```
t->next = first;
first = t;
```

Insert after a node `p`

```
t->next = p->next;
p->next = t;
```

The most important safety rule is:

> Connect the new node to the rest of the list before overwriting the old link.

The key line pair is:

```
t->next = p->next;
p->next = t;
```

If you understand those two lines deeply, you understand the heart of Module 7.